MiniDash - AI-Assisted Development Reflection

Course: CSCI 665 - Software Engineering

Project: Mini-Dash Track B: Group Options

Three Defects in AI Output

While developing the MiniDash project, I had encountered multiple problems with the AI-assisted code that required additional review and correction to make sure it was up to my standards of work.

Defect 1: Database Schema Errors

During the implementation of the groups orders database, the issue with the SQL table definitions prevented the schema from being created successfully. The problem involved the placement of constraints within the *group_members* table definition. I discovered the issue while attempting to test the constraints and executing the building of the schema. I then went in and corrected the SQL structure and reran the relevant tests and files to verify that the tables could be created and that the constraints are in line with the Architecture Design Record (ADR), demonstrating that even relatively-straightforward AI-generated SQL (even though SQL has existed since 1974), still needs to be executed and verified, rather than assuming that the AI is correct.

Defect 2: Unsafe Member Limit

An earlier approach to joining the group orders separated the member-count checks from the database insertions. While this appeared to enforce the 10-member limit, doing this introduced a concurrency risk: two members could possibly observe the same member count and both join the last position.

After reviewing the implementation, I went into the repository's logic and used SQLite's *BEGIN IMMEDIATE* transaction, allowing the app to check the group's status, expiration, and member count, before inserting the new member within the same write transaction. I went ahead and verified the correctness of the new implementation using tests for full groups, successful joins, and expiration boundaries. Although the tests can confirm the expected behavior, if I had more time or were to expand on this, I would want more dedicated concurrent-request testing to introduce even further validation.

Defect 3: Incomplete Expiration Handling

The initial expiration logic was primarily enforced when people first joined the group or modifying the group orders, causing inconsistencies with inactive groups that could still appear to be *OPEN* when the summary was requested at the end of a transaction.

I went in and identified the limitation while reviewing how the summary endpoints retrieve the group's status. I added more expiration detection to the summary repository function so that an inactive open group would be marked as *EXPIRED* before the status of the group was returned when checked for checking out an order and checking for inactivity. This reinforced the importance of testing how the different endpoints interact with each other and the same business rules.

2. Where Did AI Help the Most and Least?

AI helped me the most when implementing and testing the application's multiple different layers. It helped to assist me with organizing functionality into the repositories, services, routes, and the Pydantic models. This was really helpful as I was able to understand how to use Pydantic as I have never interacted with it before (to my knowledge.) As an example, AI helped to develop a rollback test that actively triggered a failure while inserting an order item, helping to confirm that the app did not leave behind a partially created order and added it to the orders table. It helped me provide strong evidence of the transactional accuracies, rather than simply reviewing the code to make sure everything ran without issue.

AI helped me the least when the development process became extremely fragmented and added more overhead than what was necessary. At extremely early stages (slices 1 and 2) the AI handled every function and test through many separate actions, supporting careful review but making development slower than necessary. Later, I went in and refined my prompts by suggesting we make larger, more manageable steps so the unnecessary back-and-forth was eliminated while keeping the ability to review and verify each slice.

3. Changes to My Prompting Approach

As the project progressed, my prompting approach evolved dramatically throughout the process. During the first few slices, I was more focused on small, highly specific requests to establish the project's necessary architecture and learning how the components reacted with one another.

As development progressed, evaluating the code became more comfortable and I became a little bit more comfortable about adding what AI created code-wise to the project while I focused on the complete features, including the repository operations, service logic, API routes, and

integration testing. I also became more deliberate about communicating with the AIs (ChatGPT and GitHub Copilot) about constraints. I maintained the ten-member limit, required the host only be able to lock a group, emphasize the transitions that needed to be atomic to be as such, and used a fixed delivery fee for easier testing.

By the final slice, my priorities were not simply to generate code (they never were), but making sure the code I did run properly, passed the testings I wanted, and ensuring that the checkout respected the existing schema. I also made sure that the responses the AI produced still remained compatible with the baseline MiniDash endpoints. Overall, I move from needed assistance with the individual details towards using the AI as a support.

4. A Decision I Did Not Delegate

One design decision I continued to retain control over was using the fixed *$15.95 USD delivery fee* for the group orders. Rather than implementing a dynamic fee-system, which would require actual locations and massive amounts of testing, I chose to keep a constant amount to keep the project focused on the requirements. This allowed me to concentrate on correctly diving the fee among the group members, handling the remaining cents, and ensuring that the collected shares added up to the exact total.

The AI suggested to represent the monetary values as floats. I chose to represent it as integer cents as it would avoid the floating-point rounding problems and made calculations more straightforward to test. While it may have helped and verified the calculations, the scope of the fee model was my decision.

5. Limits of My Evidence

While the final implementation passed **69 automated tests**, the result does not establish that every edge case has been verified and taken into account. One remaining area I can point out is concurrency testing. SQLite transactions were used to protect the operations like joining groups and checking out, but I didn't perform the dedicated stress tests involving many requests coming in at once. With more time, I would create the tests necessary to verify the group membership never exceeds 10 and that duplicates would create multiple orders if concurrent-requests were made.

Another limitation involved the expiration behavior. The app checks inactivity during the relevant operations, but the current design does not have a background process to expire orders automatically. I would also want to review again whether the two hour-expiration rule should apply to locked groups during checkout as the current implementation primarily enforces the expiration for open groups.

Overall, developing the minidash demonstrated how AI can be useful for accelerating software development, but it cannot replace independent judgement. To me, the most valuable parts of the process were the ones I took control off and questioning the suggestions that were generated without the full context to verify the legitimacy of them.